

# Lesson:

## Introduction to Array



# Topics

1. What is an Array?
2. Declaration of Array.
3. Array index and storing.
4. Accessing elements in an array
5. Changing values in an array
6. Delete keyword in arrays & its problem
7. Iterating of an Array item

## What is an Array?

An array in JavaScript is a data structure that stores an ordered list of elements. It can hold elements of any data type, including numbers, strings, objects, and even other arrays. Arrays are a type of object in JavaScript and have a number of built-in methods for adding, removing, and manipulating elements.

Array length property - it is used to get the length of the array, ie **array.length**

Note - The length starts its counting from one.

**Example -**

JavaScript

```
const language = ["javascript", "python", "java", "golang"]

const len = language.length
console.log("Array length =", len)
// Array length = 4
```

**Example of an array:**

JavaScript

```
let player= ["Virat Kohli", "Rohit Sharma", "Suryakumar Yadav",
"KL Rahul", "Ravindra Jadeja", "Rishabh Pant", "Shivam"];

let numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 0];

let array = ["Hello", 20, true]
```

### Why do we need Arrays in JavaScript?

Arrays in JavaScript are needed for several reasons:

- 1. Grouping related data:** Arrays allow you to group related data together and make it easier to manipulate. For example, you could use an array to store a list of names or a list of products.

2. **Storing large amounts of data:** Arrays can store large amounts of data in a single variable, making it easier to manage and manipulate the data. This can be especially useful when working with large datasets.
3. **Improving performance:** Arrays can improve performance when working with large amounts of data. Because the elements in an array are stored in consecutive memory locations, it is faster to access elements in an array than it is to access elements in other data structures, such as objects.
4. **Ease of use:** Arrays come with a number of built-in methods for adding, removing, and manipulating elements. This makes it easy to perform operations on the data stored in an array.
5. **Better readability:** Arrays make your code more readable by grouping related data together in a single structure. This makes it easier for others to understand your code and makes it easier for you to maintain your code in the future.

Overall, arrays in JavaScript are a crucial tool for managing and manipulating data in various locations. They provide a flexible and efficient way to store and process data and make writing clean, maintainable code easier.

## Declaration of Array:

There are several ways to declare an array in JavaScript.

1. **Using square brackets [ ]:** This is the most common and recommended way to declare an array in JavaScript. You can create an array by enclosing a comma-separated list of elements in square brackets:

JavaScript

```
let fruits = ["Apple", "Mango", "Banana", "Kiwi"];
```

2. **Using the Array constructor:** You can also create an array using the Array constructor. You can pass in the length of the array or a list of elements:

JavaScript

```
let numbers = new Array(1, 2, 3, 4, 5);
```

```
let emptyArray = new Array(5); // Creates an array with 5 empty elements.
```

It is generally not recommended to create an array using the Array constructor in JavaScript, due to the following reasons -

- **Inconsistent behavior** - The behavior of the Array constructor is inconsistent depending on the number and type of arguments passed to it. If you pass a single numeric argument, it is treated as the desired length of the array, and an empty array with that length is created. If you pass multiple arguments, those arguments become the elements of the array. This can lead to confusion and unexpected results.

- Simplicity and readability - Using an array literal notation ([]) to create an array is simpler and more readable. It is a widely adopted convention in JavaScript code and provides a clear and concise way to define arrays.
- Potential security risks - Using the Array constructor with a single numeric argument can lead to potential security risks if the value is not properly validated. If an attacker is able to supply a malicious value as the argument, it could result in unexpected behavior or vulnerabilities.

3. **Using an array literal:** This is a shorthand method for creating arrays that are equivalent to using square brackets:

JavaScript

```
let colors = Array("Black", "Red", "White", "Blue");
```

## Array index and storing

Indexing and storing in arrays are related concepts in JavaScript. The index of an element in an array determines its position within the array while storing in an array refers to the act of assigning a value to an element within the array.

In JavaScript, arrays are zero-indexed, meaning that the first element has an index of 0, the second element has an index of 1, and so on. To store a value in an array, you can use the square bracket notation and the index number:

JavaScript

```
let players= []; // create an empty array
fruits[0] = "Virat"; // store "Virat" at index 0
fruits[1] = "Rohit"; // store "Rohit" at index 1
fruits[2] = "Suryakumar"; // store "Suryakumar" at index 2
```

## Accessing elements in an array

Accessing elements in an array in JavaScript is done using the square bracket notation and an index number. The index number represents the position of the element within the array, starting from 0 for the first element, 1 for the second element, and so on.

For example, consider the following array:

JavaScript

```
let players = ["Virat", "Rohit", "Suryakumar"];
// To access the first element "Virat", you would use the
following code.

players[0]; // returns "Virat"
```

You can access any element in the array using its index number:

JavaScript

```
players[1]; //returns "Rohit"  
players[2]; // returns "Suryakumar"
```

It's important to keep in mind that if you try to access an index that is outside the bounds of the array, you will get undefined.

And also if you try to access a negative index it will give you an undefined.

**For example,**

JavaScript

```
players[3]; // returns undefined  
players[-1]; // returns undefined  
players[-3]; // returns undefined
```

In some other programming languages like Java, C, and C++ if you try to access a negative index, you will get an error which is `ArrayIndexOutOfBoundsException` or `Invalid OutOfRange`.

Overall, accessing elements in an array is a basic operation in JavaScript that allows you to retrieve values stored in the array. Understanding how to access elements in an array is essential for working with arrays in JavaScript.

## Changing values in an Array

In JavaScript, An Array value item can be changed by accessing the element with its index and assigning a new value to it.

**Example:**

JavaScript

```
const hobbies = ["Coding", "Learning", "Watching movies"]  
  
hobbies[0] = "building Project" // change to building Project  
hobbies[2] = "Football" // change to Football  
  
// output -  
[ 'building Project', 'Learning', 'Football' ]
```

The above example is used to change the value of an array using the index of the particular array items.

## Delete keyword in arrays & its problem

In JavaScript, the delete keyword is used to delete a property from an object, including an array element. However, using delete to remove elements from an array can lead to unexpected behavior and introduce problems.

Here are some impacts and problems associated with using the delete keyword on arrays:

- Sparse Arrays – The delete keyword does not update the length property of the array. Therefore, if you delete an element using delete, the length of the array remains the same, giving a false impression of the array's actual size.
- Array length – The delete keyword does not update the length property of the array. Therefore, if you delete an element using delete, the length of the array remains the same, giving a false impression of the array's actual size.
- Performance Impact – Deleting an element with delete does not reclaim the memory associated with that element. It simply marks the element as undefined
- Array Integrity – Using delete on an array can disrupt the integrity of the array's order and indices. The deleted element is replaced with an empty slot, and subsequent elements are not automatically shifted to fill the gap

### Example – delete in an array –

JavaScript

```
const array = [1, 2, 3, 4];
console.log(array.length); // output - 4
delete array[2];
console.log(array); // output - [ 1, 2, <1 empty item>, 4 ]
console.log(array[2]); // output - undefined
console.log(array.length); // output - 4 same length even
after delete
```

## Iterating of Array items

In JavaScript, An array of items can be iterated or looped in the following ways –

- for loop
- for...of loop
- while loop

### for loop

It allows us to iterate over an array by using the index of the array items.

#### Example -

JavaScript

```
const array = [1, 2, 3];

for (let index = 0; index < array.length; index++) {
  const element = array[index];
  console.log(element);
}

// - output -
1
2
3
```

### for... of loop

It is a newer JavaScript feature introduced in ECMAScript 6 (ES6) that allows us to iterate over iterable arrays, including objects.

#### Example

JavaScript

```
const array = [1, 2, 3];
for (const iterator of array) {
  console.log(iterator);}

// output ---
1
2
3
```

### while loop

The while loop allows us to loop over an array by checking a condition before each iteration.

#### Example -

JavaScript

```
const array = [1, 2, 4, 3];  
let i = 0;  
while (array.length > i) {  
  console.log(array[i]);  
  i++;}  
// output -  
1  
2  
3  
4
```

In the next section, we'll explore additional methods for iterating through arrays in JavaScript.

